Design Patterns

Creational · Structural · Behavioral

Modules built for another developer’s client code

Purpose

Patterns are shared answers to recurring design pressures. They help you shape module boundaries so callers get a calm, intention-revealing API while the module stays free to grow, swap internals, or add variants without rewriting every consumer.

Developer experience goal

Treat each slide as a handover story: you own the module; another developer owns the client. Good patterns make that handover kind—small surface area, clear abstractions, and room to extend the module graciously so clients keep writing the same simple calls.

How to read each pattern slide

Problem / Solution	Why the current shape hurts, and what the pattern changes about dependencies and responsibilities.
Before / After UML	Always includes a Client box. Follow dashed arrows from Client to see what caller code must know. After should depend on a narrow API, not a pile of concretes.
Client Code	How the other developer uses (or is forced to misuse) your module.
Module Code	The API boundary you hand over—the implementation clients should not need to reopen when requirements grow.

Look for fewer Client dependencies after the pattern, and for module code that stays easy to use while remaining easy to extend.

Singleton

Creational Design Pattern

Ensures a class has only one instance and provides a global point of access to it.

CASE STUDY: Shared Config across services

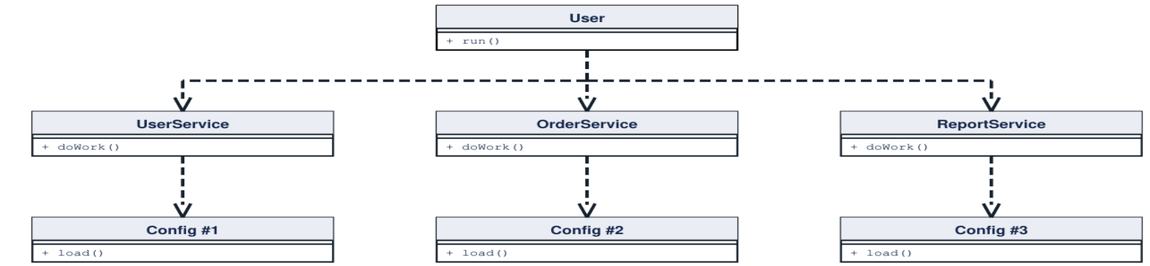
Problem

- User drives UserService / OrderService / ReportService; each needs Config.
- Call order is not fixed—any service may run first.
- Each service copies if config == null: config = new Config() → multiple instances.

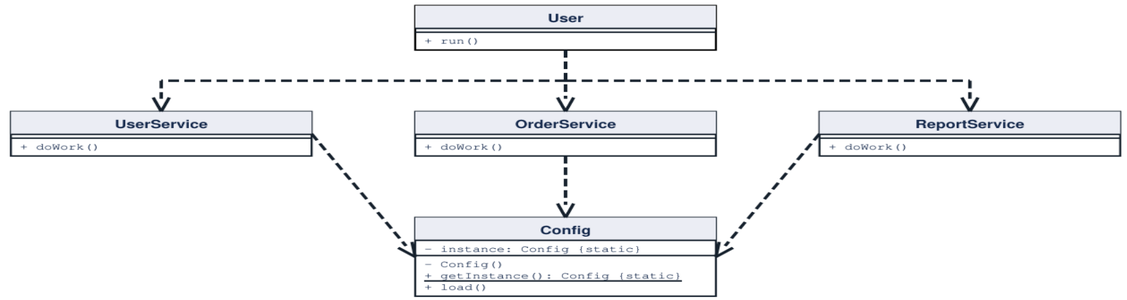
Solution

- User still calls doWork(); services only call Config.getInstance().
- The null-check and construction live once inside Config.
- First caller creates it; every service shares that instance.

Before (direct coupling)



After (using Singleton)



Service Code (Before)

Pseudocode

```
// inside each service (User API unchanged)
if config == null:
    config = new Config()
config.load()
// same guard in UserService, OrderService, ReportService
```

Service Code (After)

Pseudocode

```
// inside each service (User API unchanged)
config = Config.getInstance()
config.load()
// same load; guard moved into Config
```

Module Code (API boundary)

Pseudocode

```
// MODULE – Config singleton (owns the lazy init)
private static instance
private Config(): load()
static getInstance():
    if instance == null:
        instance = new Config()
    return instance
```

Source: https://refactoring.guru/design-patterns/singleton

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

Factory Method

Creational Design Pattern

Provides an interface for creating objects, but lets subclasses decide which class to instantiate.

CASE STUDY: Plane, truck, and ship transport

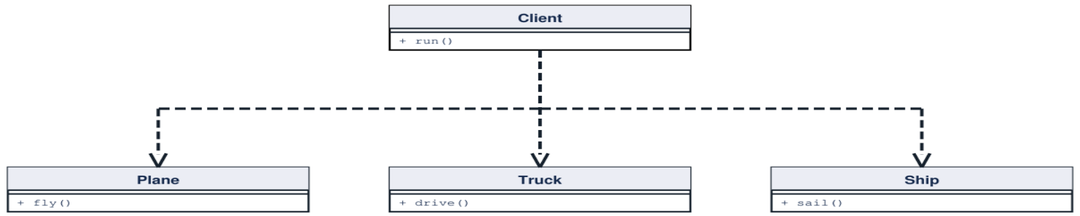
Problem

- Client code needs to create different types of objects (e.g. Plane, Truck, Ship).
- The client contains conditional logic and knows about all concrete classes.
- Adding a new type requires modifying client code (violates Open/Closed Principle).
- Object creation logic is scattered and duplicated across the application.
- Client is tightly coupled to concrete classes.

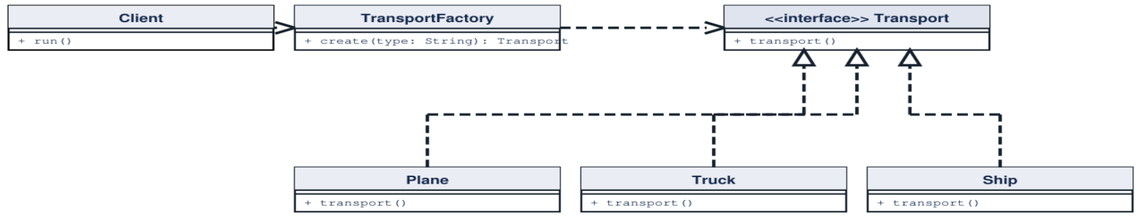
Solution

- Define a factory interface/class for creating products (e.g. Transport).
- Concrete factory decides which concrete product to instantiate.
- Client code works with the factory and the product interface, not concrete classes.
- To add a new type, add a new product and update the factory (client code unchanged).
- Centralizes and encapsulates object creation logic.

Before (direct coupling)



After (using Factory Method)



Client Code (Before)

Pseudocode

```
plane = new Plane(); plane.fly()
truck = new Truck(); truck.drive()
ship = new Ship(); ship.sail()
// three types; three different method names
```

Client Code (After)

Pseudocode

```
fleet = [factory.create("plane"),
         factory.create("truck"),
         factory.create("ship")]
for t in fleet: t.transport()
// same three results; one Transport method
```

Module Code (API boundary)

Pseudocode

```
// MODULE – creation API for client developers
// TransportFactory.create(type)
if type == "plane": return new Plane()
elif type == "truck": return new Truck()
elif type == "ship": return new Ship()
// Plane/Truck/Ship implement Transport.transport()
```

Source: https://refactoring.guru/design-patterns/factory-method

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

Abstract Factory

Creational Design Pattern

Create related product families through a factory interface.

CASE STUDY: UPS and DHL transport families

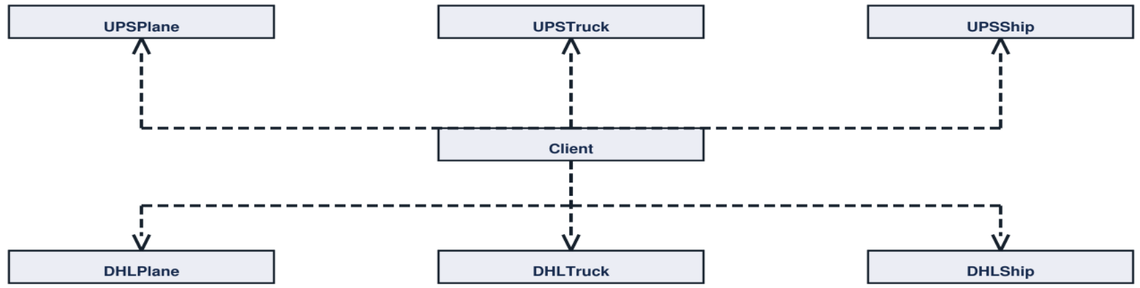
Problem

Client news every carrier concrete and calls drive / sail / fly separately.

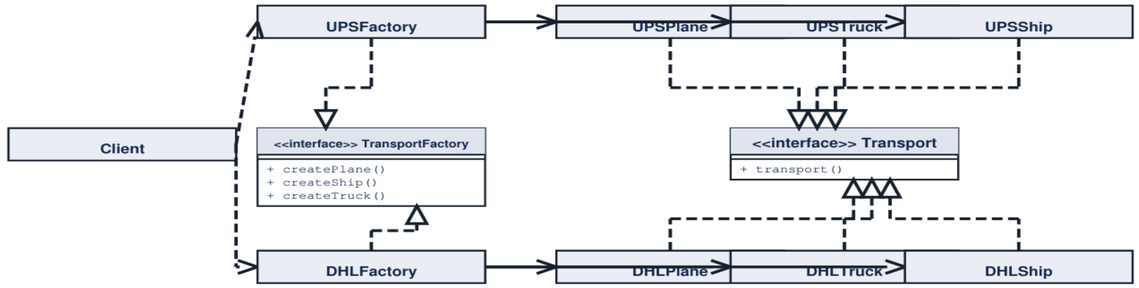
Solution

Pick a family factory; create as Transport; one transport() loop.

Before (direct coupling)



After (using Abstract Factory)



Client Code (Before)

Pseudocode

```
dt = new DHLTruck(); dt.drive()
ds = new DHLShip();  ds.sail()
dp = new DHLPlane(); dp.fly()
ut = new UPSTruck(); ut.drive()
us = new UPSShip();  us.sail()
up = new UPSPlane(); up.fly()
```

Client Code (After)

Pseudocode

```
dhl = new DHLFactory()
dp = dhl.createPlane(); ds = dhl.createShip(); dt = dhl.createTruck()
ups = new UPSFactory()
up = ups.createPlane(); us = ups.createShip(); ut = ups.createTruck()
fleet = [dp, ds, dt, up, us, ut]
for t in fleet: t.transport()
```

Source: https://refactoring.guru/design-patterns/abstract-factory

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

Builder

Creational Design Pattern

Separate construction steps from the representation being assembled.

CASE STUDY: Houses with optional features

Problem

- House grows telescoping constructors (rooms/doors/windows + pool/trees/…).
- Client passes long positional bool lists—unclear which flag is which.
- Adding a feature forces new overloads and updates every call site.

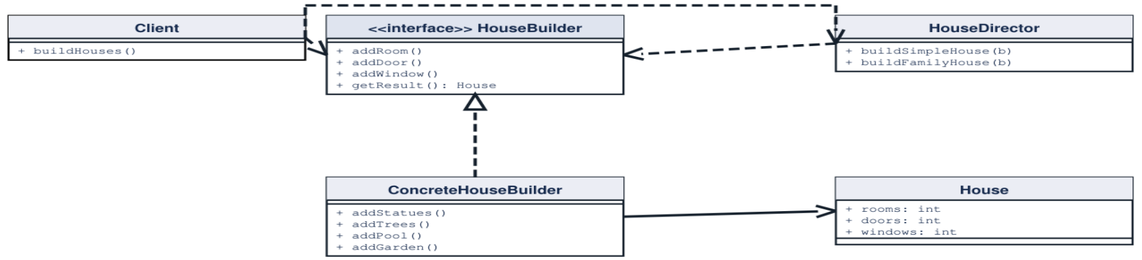
Solution

- HouseBuilder names each step; ConcreteHouseBuilder adds extras.
- HouseDirector encodes recipes (simple / family); Client can also chain steps.
- Same houses; readable construction; product is House.

Before (direct coupling)



After (using Builder)



Client Code (Before)

Pseudocode

```
houseA = new House(1, 1, 1)
houseB = new House(2, 2, 1, false, false, true, true)
houseC = new House(4, 3, 3, true, false, false, true)
houseD = new House(2, 1, 2, false, true, false, false)
houseE = new House(6, 4, 5, true, true, true, true)
// five houses; magic bool positions
```

Client Code (After)

Pseudocode

```
director = new HouseDirector()
houseA = director.buildSimpleHouse(new ConcreteHouseBuilder())
houseB = new ConcreteHouseBuilder().addWindow().addWindow()
    .addDoor().addDoor().addRoom().addStatues().addGarden().getResult()
houseC = director.buildFamilyHouse(new ConcreteHouseBuilder())
houseD = new ConcreteHouseBuilder().addWindow().addWindow()
    .addDoor().addRoom().addRoom().addTrees().getResult()
houseE = new ConcreteHouseBuilder()
    .addWindow().addWindow().addWindow().addWindow().addWindow().addWindow()
    .addDoor().addDoor().addDoor().addDoor()
    .addRoom().addRoom().addRoom().addRoom().addRoom()
    .addPool().addTrees().addStatues().addGarden().getResult()
// same five houses; named steps / director recipes
```

Source: https://refactoring.guru/design-patterns/builder

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

Adapter

Structural Design Pattern

Translate an existing interface into the interface a client expects.

CASE STUDY: XML stock data and JSON analytics

Problem

The stock app supplies XML; the analytics library requires JSON.

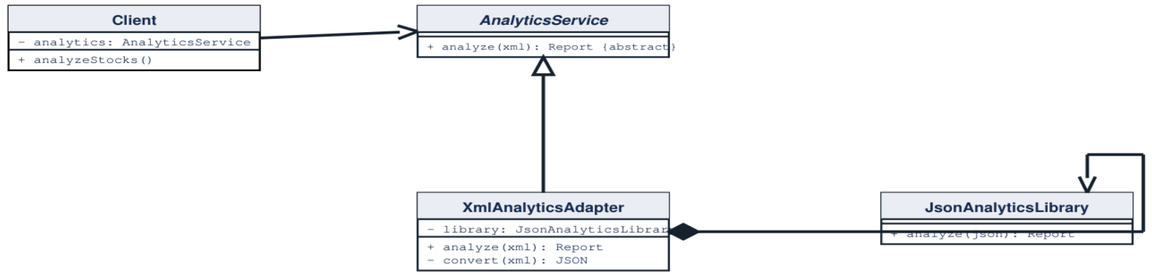
Solution

Adapter inherits AnalyticsService and composes JsonAnalyticsLibrary; converts then delegates.

Before (direct coupling)



After (using Adapter)



Client Code (Before)

Pseudocode

```
xml = provider.getStockData()
json = convertXmlToJson(xml)
report = library.analyze(json)
// same stock report; client does the conversion
```

Client Code (After)

Pseudocode

```
xml = provider.getStockData()
report = adapter.analyze(xml)
// same stock report; adapter owns the conversion
```

Module Code (API boundary)

Pseudocode

```
// MODULE — AnalyticsService API for client developers
// XmlAnalyticsAdapter extends AnalyticsService
// composes library: JsonAnalyticsLibrary
// analyze(xml):
//   json = convert(xml)
//   return library.analyze(json)
```

Source: https://refactoring.guru/design-patterns/adapter

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

Bridge

Structural Design Pattern

Separate two independent dimensions so both can evolve independently.

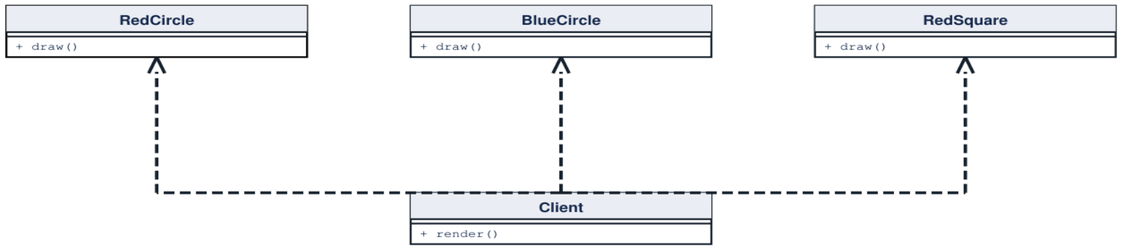
Problem

Each shape/color combination requires another subclass.

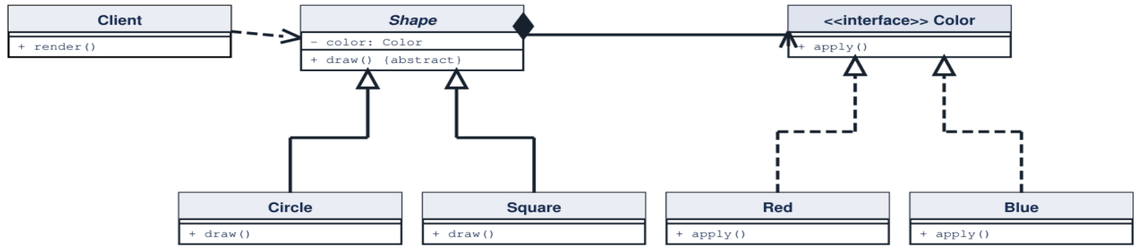
Solution

Shapes hold a Color reference instead of encoding color in inheritance.

Before (direct coupling)



After (using Bridge)



Client Code (Before)

Pseudocode

```
new RedCircle().draw()
new BlueSquare().draw()
// red circle + blue square; one class per combo
```

Client Code (After)

Pseudocode

```
shapes = [new Circle(new Red()),
          new Square(new Blue())]
for s in shapes: s.draw()
// same two drawings; compose shape × color
```

Module Code (API boundary)

Pseudocode

```
// MODULE — Shape API for client developers
// Circle.draw()
color.apply()
renderCircleGeometry()
// color and geometry vary independently
```

Source: https://refactoring.guru/design-patterns/bridge

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

Facade

Structural Design Pattern

Expose a simple entry point to a complex subsystem.

CASE STUDY: Video conversion subsystem

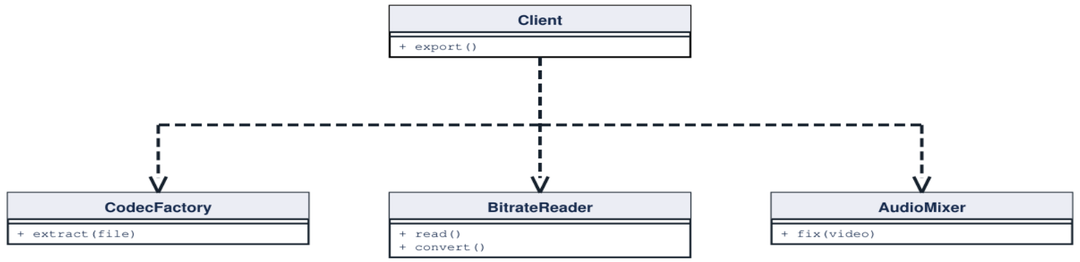
Problem

Every client coordinates codecs, bitrate conversion, and audio repair.

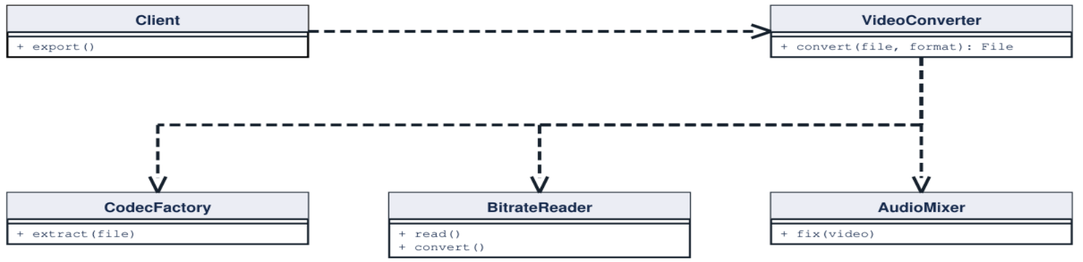
Solution

A facade orchestrates the subsystem behind one conversion call.

Before (direct coupling)



After (using Facade)



Client Code (Before)

Pseudocode

```
codec = CodecFactory.extract(file)
raw = BitrateReader.read(file, codec)
video = BitrateReader.convert(raw, mp4)
result = AudioMixer.fix(video)
// export file as mp4 via subsystem steps
```

Client Code (After)

Pseudocode

```
result = new VideoConverter().convert(file, "mp4")
// same mp4 export; one facade call
```

Module Code (API boundary)

Pseudocode

```
// MODULE – VideoConverter facade API for client developers
// VideoConverter.convert(file, format)
source = CodecFactory.extract(file)
raw = BitrateReader.read(file, source)
video = BitrateReader.convert(raw, codecFor(format))
return AudioMixer.fix(video)
```

Source: https://refactoring.guru/design-patterns/facade

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

Proxy

Structural Design Pattern

A stand-in object that controls access to a real service without changing its interface.

CASE STUDY: Expensive database service with caching

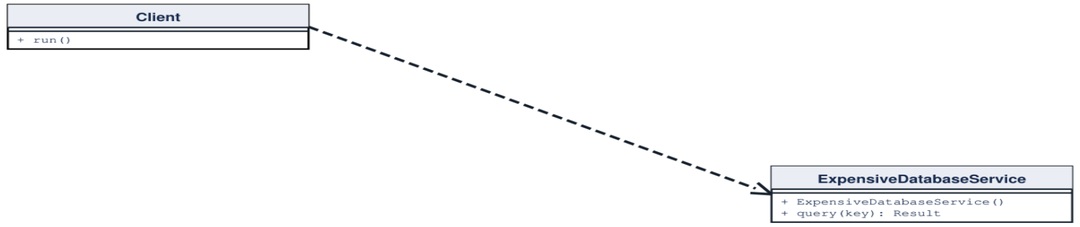
Problem

- The real service is expensive to create or use (e.g. database, remote API, large object).
- Repeated queries can be slow and waste resources.
- The client may need to handle initialization or access control logic.
- We want extra behavior (caching, logging, access control) without changing client code.

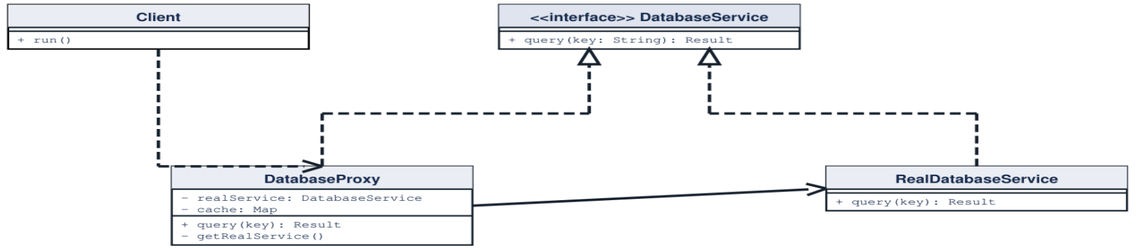
Solution

- Introduce a proxy that implements the same interface as the real service.
- The proxy controls access, adding lazy init, caching, logging, or access control.
- Client uses the same interface—only the class it instantiates changes.
- Adds: caching, lazy initialization, optional access control and logging.

Before (direct coupling)



After (using Proxy)



Client Code (Before)

Pseudocode

```
db = new ExpensiveDatabaseService()
r1 = db.query("users")
r2 = db.query("users")
// two "users" results; two expensive hits
```

Client Code (After)

Pseudocode

```
db = new DatabaseProxy()
r1 = db.query("users")
r2 = db.query("users")
// same two "users" results; second from cache
```

Module Code (API boundary)

Pseudocode

```
// MODULE — DatabaseService API for client developers
// DatabaseProxy.query(key)
if cache.contains(key): return cache[key]
if realService == null: realService = new RealDatabaseService()
result = realService.query(key)
cache[key] = result; return result
```

Source: https://refactoring.guru/design-patterns/proxy

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

Composite

Structural Design Pattern

Treat individual objects and recursive groups through one interface.

CASE STUDY: Boxes containing products and boxes

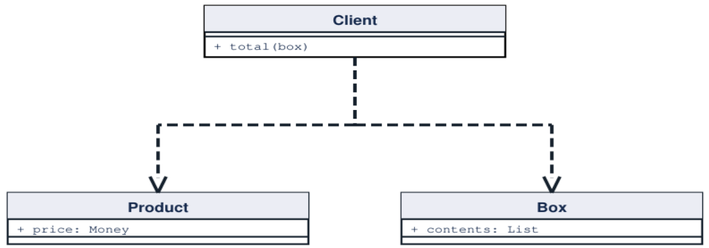
Problem

Pricing code distinguishes products from nested boxes repeatedly.

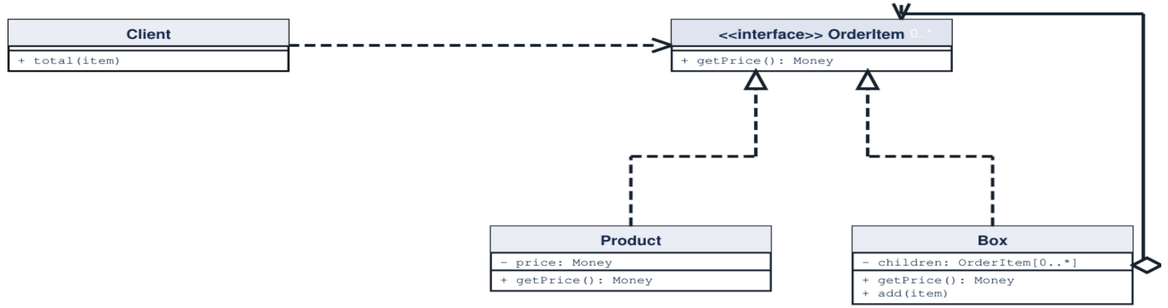
Solution

A box sums child prices; each child follows the same contract.

Before (direct coupling)



After (using Composite)



Client Code (Before)

Pseudocode

```
total = 0
for item in order.contents:
    if item is Product: total += item.price
    else: total += priceNestedBox(item)
// price of product + nested box
```

Client Code (After)

Pseudocode

```
total = order.getPrice()
// same total; one getPrice() on OrderItem
```

Module Code (API boundary)

Pseudocode

```
// MODULE — OrderItem API for client developers
// Box.getPrice()
total = 0
for child in children:
    total += child.getPrice()
return total
```

Source: https://refactoring.guru/design-patterns/composite

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

Template Method

Behavioral Design Pattern

Fix an algorithm’s order while subclasses implement selected steps.

CASE STUDY: Mining PDF and CSV documents

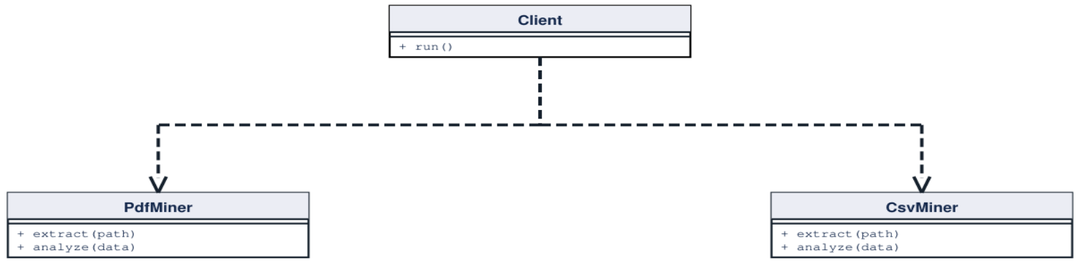
Problem

Format-specific miners duplicate the processing and reporting pipeline.

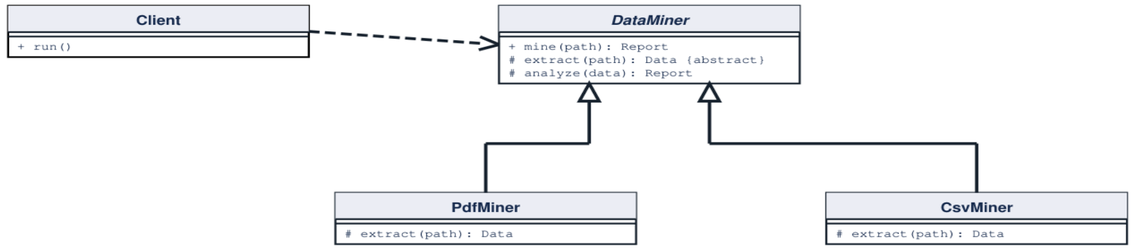
Solution

Move the sequence into a base method; override extraction only.

Before (direct coupling)



After (using Template Method)



Client Code (Before)

Pseudocode

```
pdfReport = analyze(pdf.extract(pdfPath))
csvReport = analyze(csv.extract(csvPath))
// two reports; client repeats extract+analyze
```

Client Code (After)

Pseudocode

```
miners = [new PdfMiner(), new CsvMiner()]
reports = []
for m in miners: reports.add(m.mine(pathFor(m)))
// same two reports; one mine() pipeline each
```

Module Code (API boundary)

Pseudocode

```
// MODULE — DataMiner.mine API for client developers
// DataMiner.mine(path): fixed sequence
data = extract(path)
return analyze(data)
// PdfMiner.extract(path): return parsePdf(path)
```

Source: https://refactoring.guru/design-patterns/template-method

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

Strategy

Behavioral Design Pattern

Make alternative algorithms interchangeable through a shared contract.

CASE STUDY: Navigation route algorithms

Problem

A navigator grows conditionals for road, walking, and transit routes.

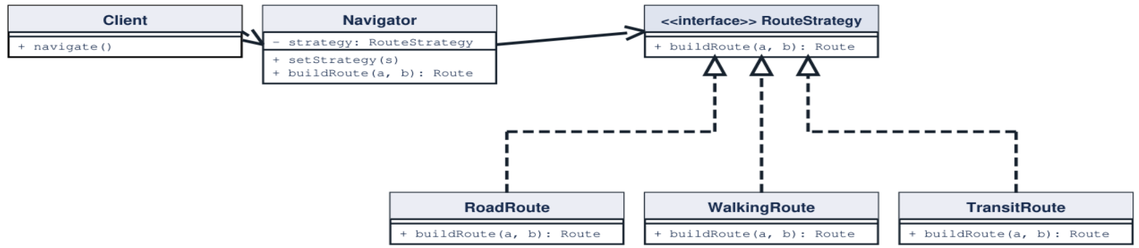
Solution

Delegate routing to a strategy selected by the client.

Before (direct coupling)



After (using Strategy)



Client Code (Before)

Pseudocode

```
road = roadRoute(a, b)
walk = walkRoute(a, b)
transit = transitRoute(a, b)
// three routes; client branches per mode
```

Client Code (After)

Pseudocode

```
for s in [new RoadRoute(), new WalkingRoute(),
          new TransitRoute()]:
    navigator.setStrategy(s)
    routes.add(navigator.buildRoute(a, b))
// same three routes; one buildRoute() API
```

Module Code (API boundary)

Pseudocode

```
// MODULE — Navigator API for client developers
// Navigator.buildRoute(a, b)
return strategy.buildRoute(a, b)
// WalkingRoute builds pedestrian paths
```

Source: https://refactoring.guru/design-patterns/strategy

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

State

Behavioral Design Pattern

Delegate behavior to an object representing the current state.

CASE STUDY: Document publication workflow

Problem

Publication rules accumulate in a switch over document status.

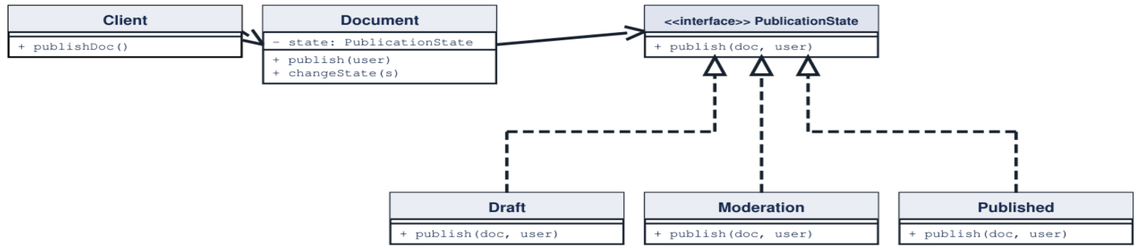
Solution

Each state handles publish and can transition the document.

Before (direct coupling)



After (using State)



Client Code (Before)

Pseudocode

```
doc.status = "draft"
doc.status = "moderation"   // submit
doc.status = "published"    // admin approve
// draft → moderation → published via status ifs
```

Client Code (After)

Pseudocode

```
doc = new Document(new Draft())
doc.publish(user)   // enters moderation
doc.publish(admin)  // becomes published
// same draft → moderation → published path
```

Module Code (API boundary)

Pseudocode

```
// MODULE — Document.publish API for client developers
// Moderation.publish(doc, user)
if user.isAdmin:
    doc.changeState(new Published())
// Published.publish(...) does nothing
```

Source: https://refactoring.guru/design-patterns/state

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

Observer

Behavioral Design Pattern

Notify registered subscribers when a publisher emits an event.

CASE STUDY: Editor events notify logging and email

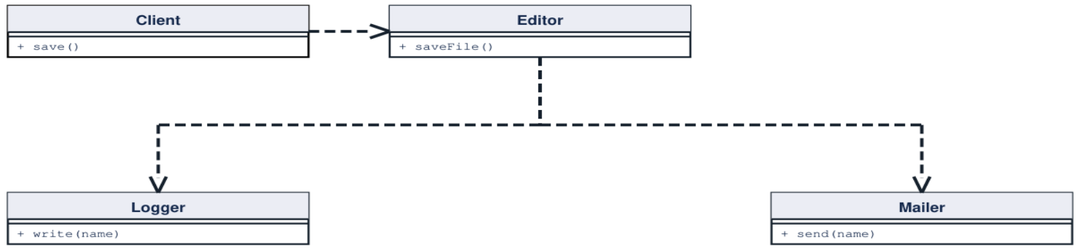
Problem

Editor logic directly invokes concrete logging and email services.

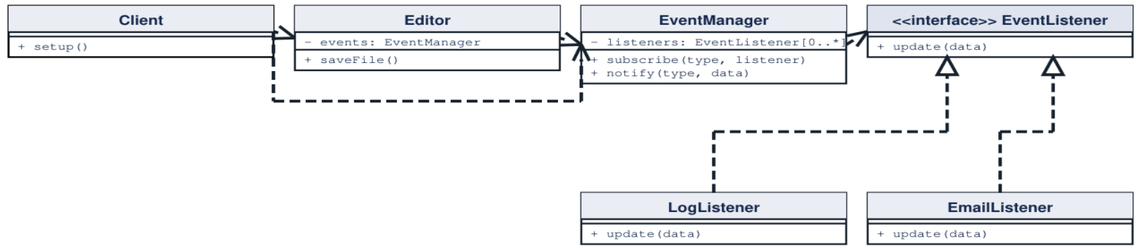
Solution

Publish an event; subscribers handle their own reactions.

Before (direct coupling)



After (using Observer)



Client Code (Before)

Pseudocode

```
editor.saveFile()
logger.write(file.name)
mailer.send(file.name)
// save + log + email; client wires reactions
```

Client Code (After)

Pseudocode

```
events.subscribe("save", new LogListener())
events.subscribe("save", new EmailListener())
editor.saveFile()
// same save + log + email; listeners react
```

Module Code (API boundary)

Pseudocode

```
// MODULE — EventManager API for client developers
// EventManager.notify(type, data)
for listener in listeners[type]:
    listener.update(data)
// Editor.saveFile(): write(); events.notify("save", name)
```

Source: https://refactoring.guru/design-patterns/observer

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency

Command

Behavioral Design Pattern

Represent a request as an object that can be invoked and undone.

CASE STUDY: Undoable text-editor actions

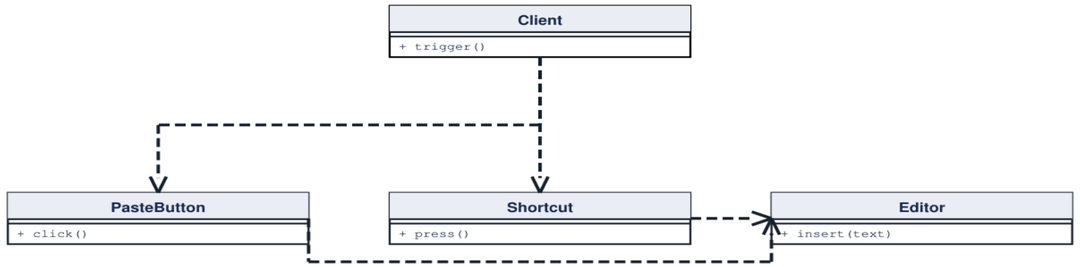
Problem

Buttons and shortcuts duplicate editing actions without shared undo.

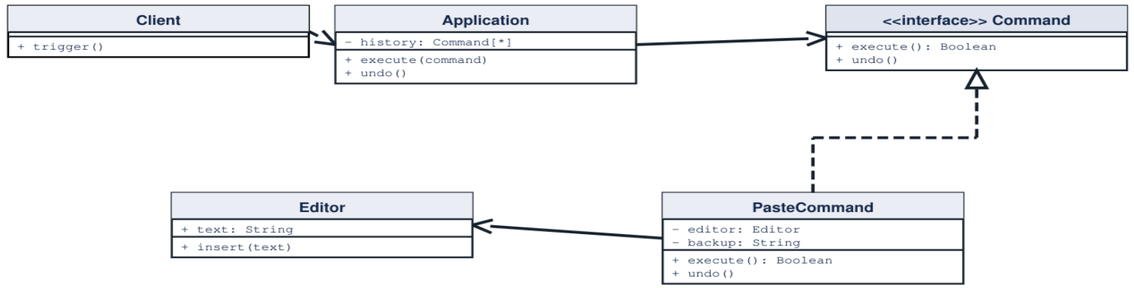
Solution

Bind both triggers to a command; store successful edits in history.

Before (direct coupling)



After (using Command)



Client Code (Before)

Pseudocode

```
pasteButton.click()  // editor.insert(text)
shortcut.press()     // editor.insert(text)
// paste from two triggers; logic duplicated, no undo
```

Client Code (After)

Pseudocode

```
paste = new PasteCommand(editor, text)
pasteButton.bind(() => app.execute(paste))
shortcut.bind(() => app.execute(paste))
pasteButton.click(); shortcut.press()
// same two pastes via shared command
```

Module Code (API boundary)

Pseudocode

```
// MODULE — Command API for client developers
// PasteCommand.execute()
backup = editor.text; editor.insert(text)
return true  // app records command in history
// undo(): editor.text = backup
```

Source: https://refactoring.guru/design-patterns/command

UML: Client box = caller code; hollow triangle = inheritance / realization; dashed open arrow = dependency